

Longest Common Subarray

Problem Statement

Given two arrays, find the longest common contiguous subarray.

Variants

- Return length only vs subarray

- Mer

How to Leverage the Solution

Use rolling DP; track end index to rebuild the subarray if needed.

Java Solution

Below is a concise reference implementation. Full comments omitted for brevity.

```
import java.util.*;

public final class LongestCommonSubarray {
    public static int length(int[] a, int[] b) {
        int n = a.length, m = b.length; int[] dp = new int[m + 1]; int best = 0;
        for (int i = 1; i <= n; i++) {
            int prev = 0;
            for (int j = 1; j <= m; j++) {
                int temp = dp[j];
                if (a[i - 1] == b[j - 1]) { dp[j] = prev + 1; if (dp[j] > best) best = dp[j]; }
                else dp[j] = 0;
                prev = temp;
            }
        }
        return best;
    }

    public static int[] subarray(int[] a, int[] b) {
        int n = a.length, m = b.length; int[] dp = new int[m + 1]; int best = 0, endA = -1;
        for (int i = 1; i <= n; i++) {
            int prev = 0;
            for (int j = 1; j <= m; j++) {
                int temp = dp[j];
                if (a[i - 1] == b[j - 1]) { dp[j] = prev + 1; if (dp[j] > best) { best = dp[j]; endA = i - 1; }
            }
            else dp[j] = 0;
            prev = temp;
        }
        if (best == 0) return new int[0];
        return Arrays.copyOfRange(a, endA - best + 1, endA + 1);
    }
}
```